硬件描述语言与 FPGA 应用实验报告

（实验：电子时钟的设计）

实验内容

实现一个电子时钟的基本功能。

要求：能够正确显示时分秒；能够设置时钟为当前正确时间。

在此基础上，请自由发挥实现更多功能以获得额外加分。

一、基本要求（90 分）：

1. **写出实验方案；**
2. **编写 Verilog HDL 代码，并综合出电路。**
3. **使用 waveform 仿真出波形。**
4. **编写 testbench 代码，并仿真出波形**
5. **烧录进 FPGA 开发板，并正确实现功能**

自由发挥（10 分）

（例：闹铃、定时功能、日期显示……）

1、实验方案（10 分）（Markdown）

```
## 1.1 设计概述


### 1.1.1 项目背景
数字时钟是日常生活中最常见的电子设备之一。本项目基于 FPGA（现场可编程门阵列）技术，采用 Verilog HDL 硬件描述语言，设计并实现了一款功能完善的数字时钟系统。该系统不仅具备基本的计时功能，还扩展了闹钟、整点报时等实用功能。
### 1.1.2 设计目标
- 实现 24 小时制数字时钟的基本计时功能（时:分:秒）
- 通过 8 位数码管显示时间，格式为 HH-MM-SS
- 支持按键调整时间（秒、分、时分别可调）
- 实现闹钟功能，可设置闹钟时间并在指定时间触发蜂鸣器报警
- 实现整点报时功能，每小时整点时蜂鸣器提示
- 采用 74HC595 移位寄存器驱动数码管，减少 IO 占用
### 1.1.3 开发环境
| 项目 | 说明 |
|------|------|
| 目标器件 | EP4CE10F17C8 (Altera Cyclone IV) |
```

| 开发工具 | Quartus II 13.0 |

| 系统时钟 | 50MHz 晶振 |

| 编程语言 | Verilog HDL |

1.2 系统架构设计

1.2.1 系统整体框图

```
                    ┌──────────────────────────────────────┐
                    │              top (顶层模块)              │
                    │                                      │
    ┌───────┐       │  ┌──────────────────────────────┐    │
    │ 50MHz │──Clk──│──│       Project_Segled2        │    │
    │ 晶振  │       │  │      (数字时钟核心模块)          │    │
    └───────┘       │  │                              │    │
                    │  │  ┌────────┐  ┌──────────┐    │    │
    ┌───────┐       │  │  │ 计时模块 │  │ 数码管驱动  │    │    │
    │ RST_N │───────│──│  │ 时/分/秒 │  │ 段码/位选  │    │
    │ 复位  │       │  │  └────────┘  └──────────┘    │    │
    └───────┘       │  │                              │    │
                    │  │  ┌────────┐  ┌──────────┐    │    │
    ┌───────┐       │  │  │ 按键消抖 │  │ 闹钟/整点  │    │    │     ┌───────┐
    │ KEY1  │───────│──│  │ 模式切换 │  │ 报时控制   │────│────│────▶│ 蜂鸣器 │
    │ KEY2  │───────│──│  └────────┘  └──────────┘    │    │     │ BEEP  │
    └───────┘       │  └──────────────────────────────┘    │     └───────┘
                    │                  │                   │
                    │                  ▼                   │
                    │  ┌──────────────────────────────┐    │     ┌───────┐   ┌───────┐
                    │  │        HC595_Driver          │    │     │ 74HC595│   │ 8 位  │
                    │  │   (74HC595 移位寄存器驱动)       │────│────▶│ 芯片  │──▶│ 数码管 │
                    │  │   SH_CP / ST_CP / DS         │    │     └───────┘   └───────┘
                    │  └──────────────────────────────┘    │
                    └──────────────────────────────────────┘
```

1.2.2 模块划分

本设计采用模块化设计思想，共分为三个模块：

| 模块名称 | 功能描述 | 文件名 |

|----------|----------|--------|

| **top** | 顶层模块，负责各子模块的连接与信号分配 | top.v |

| **Project_Segled2** | 数字时钟核心模块，包含计时、显示、按键处理、闹钟等功能 | Project_Segled2.v |

| **HC595_Driver** | 74HC595 移位寄存器驱动，实现串行数据输出 | HC595_Driver.v |

1.3 模块详细设计

1.3.1 顶层模块 (top.v)

1.3.1.1 端口定义

```verilog
module top(
    input  Clk,      // 50MHz 系统时钟
    input  Rst_n,    // 复位信号（低电平有效）
    input  KEY1,     // 模式切换/长按进入闹钟设置
    input  KEY2,     // 数值递增按键
    output SH_CP,    // 74HC595 移位时钟
    output ST_CP,    // 74HC595 锁存时钟
    output DS,       // 74HC595 串行数据
    output BEEP      // 蜂鸣器输出
);
```

1.3.1.2 功能说明

顶层模块作为系统的总控制器，主要完成以下功能：

- 将外部时钟和复位信号分配给各子模块
- 连接数字时钟核心模块与 74HC595 驱动模块
- 将数码管段码(SEG_DATA)和位选(SEG_EN)信号打包后送入 HC595 驱动

1.3.1.3 模块例化

```verilog
// 数字时钟核心模块例化
Project_Segled2 Project_Segled2_inst(
    .CLK_50M  (Clk),
    .RST_N    (Rst_n),
    .SEG_EN   (SEG_EN),      // 8 位数码管位选
    .SEG_DATA (SEG_DATA),    // 8 位数码管段码
    .FLAG1    (KEY1),
    .FLAG2    (KEY2),
    .BEEP     (BEEP)
);
// 74HC595 驱动模块例化
HC595_Driver HC595_Driver(
    .Clk   (Clk),
    .Rst_n (Rst_n),
    .Data  ({SEG_DATA, SEG_EN}),  // 16 位数据：高 8 位段码，低 8 位位选
    .S_EN  (1'b1),
    .SH_CP (SH_CP),
    .ST_CP (ST_CP),
    .DS    (DS)
);
```

1.3.2 数字时钟核心模块 (Project_Segled2.v)

1.3.2.1 端口定义

端口名	方向	位宽	描述
CLK_50M	Input	1	50MHz 系统时钟
RST_N	Input	1	低电平复位信号
FLAG1	Input	1	模式切换按键
FLAG2	Input	1	数值递增按键
SEG_EN	Output	8	数码管位选信号
SEG_DATA	Output	8	数码管段码信号
BEEP	Output	1	蜂鸣器控制信号

1.3.2.2 核心功能模块

(1) 按键消抖模块

采用计数器消抖方式，消抖时间约 20ms：

```verilog
parameter DB_CNT_MAX = 20'd100; // 消抖计数最大值
// 两级同步 + 计数消抖
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        k1_sync0 <= 1'b1; k1_sync1 <= 1'b1;
    end else begin
        k1_sync0 <= FLAG1; k1_sync1 <= k1_sync0;
    end
end
```

设计特点：

- 采用两级触发器同步，防止亚稳态
- 检测高→低电平翻转，产生单周期脉冲
- 支持长按检测（1 秒），用于进入闹钟设置模式

(2) 计时模块

基于 50MHz 时钟进行分频，实现秒、分、时的计时：

```verilog
// 关键参数
parameter SET_TIME_1S = 'd50_000_000;  // 1 秒计数值
parameter SIXTY = 'd60;                // 60 进制
parameter TWENTYFORE = 'd24;           // 24 进制
// 1 秒定时器
always @ (posedge CLK_50M or negedge RST_N) begin
    if(!RST_N)
        cnt_1s <= 28'd0;
    else if(set_mode != 2'd0)  // 调整模式下暂停
        cnt_1s <= 28'd0;
    else if(cnt_1s == SET_TIME_1S - 1)
        cnt_1s <= 28'd0;
    else
        cnt_1s <= cnt_1s + 28'b1;
```

```
end
```

计时进位逻辑：

- 秒：0-59 循环，满 60 进位
- 分：0-59 循环，满 60 进位
- 时：0-23 循环，满 24 复位

(3) 时间调整模式状态机

通过按键切换四种工作模式：

模式值	模式名称	功能描述
0	正常模式	时钟正常走时
1	调秒模式	秒位闪烁，可调整秒
2	调分模式	分位闪烁，可调整分
3	调时模式	时位闪烁，可调整时

```verilog
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N)
        set_mode <= 2'd0;
    else if(mode_pulse && !alarm_set_mode && !alarm_triggered)
        set_mode <= set_mode + 2'd1;  // 0→1→2→3→0 循环
end
```

(4) 闹钟功能模块

闹钟设置流程：

1. 长按 KEY1（1 秒）进入闹钟设置模式
2. 短按 KEY1 切换时/分设置位置
3. 按 KEY2 递增当前设置值
4. 再次长按 KEY1 退出设置并使能闹钟

```verilog
// 闹钟触发检测
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N)
        alarm_triggered <= 1'b0;
    else if(alarm_triggered && (mode_pulse || inc_pulse))
        alarm_triggered <= 1'b0;  // 任意按键取消
    else if(alarm_enabled && !alarm_set_mode &&
            cnt_h == alarm_h && cnt_m == alarm_m && cnt_s == 6'd0)
        alarm_triggered <= 1'b1;  // 时间匹配，触发闹钟
end
```

闹钟特性：

- 触发时蜂鸣器持续响、显示快速闪烁（125ms 周期）
- 任意按键可取消闹钟

- 超时 10 秒自动停止

(5) 整点报时模块

每小时整点（分秒都为 0）时触发报时：

```verilog
// 整点检测条件
if(cnt_s == 6'd0 && cnt_m == 6'd0 && cnt_1s == SET_TIME_1S - 1)
    chime_active <= 1'b1;  // 激活整点报时
```

报时特性：

- 闪烁 6 次（300ms 周期）
- 蜂鸣器间歇发声

(6) 数码管显示模块

采用动态扫描方式驱动 8 位数码管：

```verilog
// 显示格式: HH-MM-SS
// 位置分配: 7-6-5-4-3-2-1-0
//          时十位-时个位-横线-分十位-分个位-横线-秒十位-秒个位
// 1ms 扫描周期
parameter SET_TIME_1MS = 16'd50_000;
// 段码查找表（共阴极数码管）
parameter SEG_CODE_0 = 8'b1100_0000,
          SEG_CODE_1 = 8'b1111_1001,
          ...
          SEG_CODE_9 = 8'b1001_0000,
          SEG_CODE_DASH = 8'b1011_1111;  // 横线"-"
```

二进制转 BCD 算法（Double Dabble）：

```verilog
// 将 6 位二进制数转换为 2 位 BCD
for(i = 5; i >= 0; i = i - 1) begin
    if (s_ones >= 4'd5) s_ones = s_ones + 4'd3;
    if (s_tens >= 4'd5) s_tens = s_tens + 4'd3;
    s_tens = {2'b00, s_tens[2:0], s_ones[3]};
    s_ones = {2'b00, s_ones[2:0], cnt_s[i]};
end
```

(7) 蜂鸣器控制模块

生成约 2kHz 方波驱动蜂鸣器：

```verilog
parameter BEEP_FREQ_CNT = 16'd12500;  // 50MHz/12500/2 = 2kHz
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N)
        beep_cnt <= 16'd0;
    else if(beep_cnt >= BEEP_FREQ_CNT - 1) begin
```

```
        beep_cnt <= 16'd0;
        beep_tone <= ~beep_tone;
    end else
        beep_cnt <= beep_cnt + 16'd1;
end
```

1.3.3 74HC595 驱动模块 (HC595_Driver.v)

1.3.3.1 74HC595 芯片简介

74HC595 是一款 8 位串入并出移位寄存器，具有以下特点：

- 串行数据输入，8 位并行数据输出
- 具有输出锁存功能
- 可级联扩展

引脚功能：

引脚	名称	功能
DS	数据输入	串行数据输入端
SH_CP	移位时钟	上升沿移入数据
ST_CP	锁存时钟	上升沿锁存数据到输出

1.3.3.2 驱动时序设计

```verilog
parameter DATA_WIDTH = 16;  // 16 位数据（8 位段码 + 8 位位选）
parameter CNT_MAX = 4;      // 分频系数
// 移位时钟生成
always@(posedge Clk or negedge Rst_n)
    if(!Rst_n)
        divider_cnt <= 16'd0;
    else if(divider_cnt == CNT_MAX)
        divider_cnt <= 16'd0;
    else
        divider_cnt <= divider_cnt + 1'b1;
assign sck_pluse = (divider_cnt == CNT_MAX);
```

时序状态机（32 个状态）：

```verilog
case(SHCP_EDGE_CNT)
    5'd0: begin SH_CP <= 1'b0; ST_CP <= 1'b1; DS <= r_data[15]; end
    5'd1: begin SH_CP <= 1'b1; ST_CP <= 1'b0; end
    5'd2: begin SH_CP <= 1'b0; DS <= r_data[14]; end
    5'd3: begin SH_CP <= 1'b1; end
    // ... 依次移出 16 位数据
    5'd30: begin SH_CP <= 1'b0; DS <= r_data[0]; end
    5'd31: begin SH_CP <= 1'b1; end
endcase
```

```
```

时序图：

```
        ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐     ┌─┐ ┌─┐
SH_CP ──┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─┘ └─ ... ─┘ └─┘ └──

      ┌─┐                                       ┌─┐
ST_CP ┘ └──────────────────────────────── ... ────┘ └

      ──┬───┬───┬───┬───┬───┬───┬───┬── ... ──┬───┬───
DS      D15 D14 D13 D12 D11 D10 D9  D8      D1  D0
```

1.4 系统功能说明

1.4.1 操作说明

操作	按键	功能说明
模式切换	短按 KEY1	正常→调秒→调分→调时→正常 循环
数值调整	按 KEY2	在调整模式下，递增当前选中位
进入闹钟设置	长按 KEY1（1 秒）	进入闹钟时间设置模式
切换闹钟设置位	短按 KEY1	在闹钟设置模式下切换时/分
设置闹钟值	按 KEY2	递增闹钟时间
退出闹钟设置	长按 KEY1（1 秒）	保存并使能闹钟
取消闹钟	按任意键	闹钟响时按任意键停止

1.4.2 显示格式

```
  ┌───┬───┬───┬───┬───┬───┬───┐
  │ H │ H │ - │ M │ M │ - │ S │ S │
  │ 时 │ 时 │   │ 分 │ 分 │   │ 秒 │ 秒 │
  │十位│个位│   │十位│个位│   │十位│个位│
  └───┴───┴───┴───┴───┴───┴───┘
```

1.4.3 状态指示

状态	显示效果
正常计时	全部数字稳定显示
调秒模式	秒位闪烁（250ms 周期）
调分模式	分位闪烁（250ms 周期）
调时模式	时位闪烁（250ms 周期）
闹钟设置	被调整位闪烁，秒显示 00
整点报时	全部闪烁 6 次（300ms 周期），蜂鸣器间歇响
闹钟触发	全部快速闪烁（125ms 周期），蜂鸣器持续响

1.5 资源占用分析

1.5.1 逻辑资源估算

| 资源类型 | 估算数量 | 说明 |

```
|----------|----------|------|
| 寄存器 | ~200 | 计数器、状态机、按键消抖等 |
| 组合逻辑 | ~500 LUTs | BCD 转换、段码查找、控制逻辑 |
| I/O 引脚 | 7 | Clk, Rst_n, KEY1, KEY2, SH_CP, ST_CP, DS, BEEP |
### 1.5.2 时序性能
- 系统时钟：50MHz
- 移位时钟：50MHz / 5 = 10MHz
- 数码管刷新率：1kHz（1ms 周期扫描 8 位）
- 计时精度：1 秒 = 50,000,000 个时钟周期
---
## 1.6 设计特点与创新
### 1.6.1 设计亮点
 1.6.1.1 **模块化设计**：采用层次化设计，各模块功能独立，便于调试和维护
1.6.1.2 **完善的按键处理**：
    - 两级同步防亚稳态
    - 计数器消抖（约 20ms）
    - 支持短按/长按检测
1.6.1.3 **丰富的功能扩展**：
    - 闹钟功能（可设置、可取消、超时自动停止）
    - 整点报时功能
    - 调整模式下闪烁提示
1.6.1.4 **节省 IO 资源**：使用 74HC595 串行接口，仅需 3 个 IO 即可驱动 8 位数码管
1.6.1.5  **友好的用户交互**：
    - 闪烁指示当前操作位
    - 不同模式有不同的视觉反馈
```

2.1、Verilog HDL 代码（20 分）

全部设计代码如下

顶层模块

```
module top(
    Clk,
    Rst_n,

    SH_CP,
    ST_CP,
    DS,

    KEY1,
    KEY2,

    BEEP
);
    input Clk;  //50M
    input Rst_n;

    //PHY PIN
    output SH_CP;   //shift clock
    output ST_CP;   //latch data clock
    output DS;     //shift serial data
    input KEY1; //加分钟
    input KEY2; //加小时

    output BEEP;  //蜂鸣器输出

    wire [7:0] SEG_EN;     //数码管位选（选择当前要显示的数码管）
    wire [7:0] SEG_DATA;   //数码管段选（当前要显示的内容）
Project_Segled2 Project_Segled2_inst
(
        .CLK_50M     (Clk)      ,               //时钟的端口,开发板用的 50M 晶振
        .RST_N       (Rst_n)    ,               //复位的端口,低电平复位
        .SEG_EN      (SEG_EN)   ,               //数码管使能端口
        .SEG_DATA    (SEG_DATA) ,              //数码管数据端口(查看管脚分配文档或者原理图)
          .FLAG1       (KEY1)    ,
          .FLAG2       (KEY2)    ,
          .BEEP        (BEEP)                 //蜂鸣器输出
);  HC595_Driver HC595_Driver(
        .Clk(Clk),
        .Rst_n(Rst_n),
        .Data({SEG_DATA,SEG_EN}),
        .S_EN(1'b1),
        .SH_CP(SH_CP),
```

```
        .ST_CP(ST_CP),
        .DS(DS)
    );
endmodule
```

HC595 驱动模块

```
/**************************************************
*   Module Name     :   HC595_Driver
*   Target Device   :   EP4CE10F17C8
*   Tool versions   :   Quartus II 13.0
*   Create Date     :   2017-3-31
*   Revision           :    v1.0
*   Description     :  74HC595 移位寄存器驱动设计
**************************************************/

module HC595_Driver(
        Clk,
        Rst_n,
        Data,
        S_EN,
        SH_CP,
        ST_CP,
        DS
    );
```

```
    parameter DATA_WIDTH = 16;
```

```
    input Clk;
    input Rst_n;
    input [DATA_WIDTH-1 : 0] Data;  //data to send
    input S_EN; //send en
    output reg SH_CP;   //shift clock
    output reg ST_CP;   //latch data clock
    output reg DS;  //shift serial data

    parameter CNT_MAX = 4;



    reg [15:0] divider_cnt;//分频计数器
    wire sck_pluse;

    reg [4:0]SHCP_EDGE_CNT;//SH_CP EDGE counter
```

```
reg [15:0]r_data;

always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    r_data <= 16'd0;
else if(S_EN)
    r_data <= Data;
else
    r_data <= r_data;

//clock divide
always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    divider_cnt <= 16'd0;
else if(divider_cnt == CNT_MAX)
    divider_cnt <= 16'd0;
else
    divider_cnt <= divider_cnt + 1'b1;

assign sck_pluse = (divider_cnt == CNT_MAX);

always@(posedge Clk or negedge Rst_n)
if(!Rst_n)
    SHCP_EDGE_CNT <= 5'd0;
else if(sck_pluse)begin
    if(SHCP_EDGE_CNT ==  5'd31)
        SHCP_EDGE_CNT <= 5'd0;
    else
        SHCP_EDGE_CNT <= SHCP_EDGE_CNT + 1'd1;
end
else
    SHCP_EDGE_CNT <= SHCP_EDGE_CNT;

always@(posedge Clk or negedge Rst_n)
if(!Rst_n)begin
    SH_CP <= 1'b0;
    ST_CP <= 1'b0;
    DS <= 1'b0;
end
else begin
    case(SHCP_EDGE_CNT)
        5'd0:begin SH_CP <= 1'b0; ST_CP <= 1'b1; DS <= r_data[15]; end
        5'd1:begin SH_CP <= 1'b1; ST_CP <= 1'b0;end
```

```
            5'd2:begin SH_CP <= 1'b0; DS <= r_data[14];end
            5'd3:begin SH_CP <= 1'b1; end
            5'd4:begin SH_CP <= 1'b0; DS <= r_data[13];end
            5'd5:begin SH_CP <= 1'b1; end
            5'd6:begin SH_CP <= 1'b0; DS <= r_data[12];end
            5'd7:begin SH_CP <= 1'b1; end
            5'd8:begin SH_CP <= 1'b0; DS <= r_data[11];end
            5'd9:begin SH_CP <= 1'b1; end
            5'd10:begin SH_CP <= 1'b0; DS <= r_data[10];end
            5'd11:begin SH_CP <= 1'b1; end
            5'd12:begin SH_CP <= 1'b0; DS <= r_data[9];end
            5'd13:begin SH_CP <= 1'b1; end
            5'd14:begin SH_CP <= 1'b0; DS <= r_data[8];end
            5'd15:begin SH_CP <= 1'b1; end
            5'd16:begin SH_CP <= 1'b0; DS <= r_data[7];end
            5'd17:begin SH_CP <= 1'b1; end
            5'd18:begin SH_CP <= 1'b0; DS <= r_data[6];end
            5'd19:begin SH_CP <= 1'b1; end
            5'd20:begin SH_CP <= 1'b0; DS <= r_data[5];end
            5'd21:begin SH_CP <= 1'b1; end
            5'd22:begin SH_CP <= 1'b0; DS <= r_data[4];end
            5'd23:begin SH_CP <= 1'b1; end
            5'd24:begin SH_CP <= 1'b0; DS <= r_data[3];end
            5'd25:begin SH_CP <= 1'b1; end
            5'd26:begin SH_CP <= 1'b0; DS <= r_data[2];end
            5'd27:begin SH_CP <= 1'b1; end
            5'd28:begin SH_CP <= 1'b0; DS <= r_data[1];end
            5'd29:begin SH_CP <= 1'b1; end
            5'd30:begin SH_CP <= 1'b0; DS <= r_data[0];end
            5'd31:begin SH_CP <= 1'b1; end
        endcase
    end
endmodule
```

数字时钟功能模块

```
//-----------------------------------------------------------------------------
//--    文件名      :   Project_Segled2.v
//--    描述        :   数字时钟设计(含闹钟功能)
//-----------------------------------------------------------------------------
module Project_Segled2
(
    //输入端口
    CLK_50M,RST_N,
    //输出端口
    SEG_DATA,SEG_EN,

    FLAG1,FLAG2,

    //蜂鸣器输出
    BEEP
);

//-----------------------------------------------------------------------------
//--    外部端口声明
//-----------------------------------------------------------------------------
input               CLK_50M;                    //时钟的端口,开发板用的 50M 晶振
input               RST_N;                    //复位的端口,低电平复位
output reg  [ 7:0]  SEG_EN;             //数码管使能端口
output reg  [ 7:0]  SEG_DATA;           //数码管数据端口(查看管脚分配文档或者原理图)
input             FLAG1;                    //切换模式按键
input             FLAG2;                    //加数按键
output reg        BEEP;                 //蜂鸣器输出

//-----------------------------------------------------------------------------
//--  按键消抖与单击脉冲（默认按键低电平按下，50MHz 时钟，约 20ms 消抖）
//-----------------------------------------------------------------------------
parameter DB_CNT_MAX = 20'd100; // 20ms @50MHz
// 前向声明用于整点报时
parameter SET_TIME_1S_FWD = 'd50_000_000;
//设置定时器的时间为 1ms,计算方法为  (1*10^3)us / (1/50)us  50MHz 为开发板晶振
parameter SET_TIME_1MS = 16'd50_000;
parameter SET_TIME_1S = 'd50_000_000;
parameter SIXTY = 'd60;
parameter TWENTYFORE ='d24;
parameter state_max_yellow = 4'd0;
parameter SEG_CODE_0 = 8'b1100_0000,
          SEG_CODE_1 = 8'b1111_1001,
          SEG_CODE_2 = 8'b1010_0100,
          SEG_CODE_3 = 8'b1011_0000,
```

```
            SEG_CODE_4 = 8'b1001_1001,
            SEG_CODE_5 = 8'b1001_0010,
            SEG_CODE_6 = 8'b1000_0010,
            SEG_CODE_7 = 8'b1111_1000,
            SEG_CODE_8 = 8'b1000_0000,
            SEG_CODE_9 = 8'b1001_0000,
            SEG_CODE_DASH = 8'b1011_1111;  // 横线 "-" 只点亮 g 段
reg k1_sync0, k1_sync1, k1_level, k1_level_prev; reg [19:0] k1_cnt; reg mode_pulse;
reg k2_sync0, k2_sync1, k2_level, k2_level_prev; reg [19:0] k2_cnt; reg inc_pulse;
// 时钟计数器前向声明
reg [27:0] cnt_1s;
reg [5:0]  cnt_s;
reg [5:0]  cnt_m;
reg [5:0]  cnt_h;  // 前向声明小时计数器
// 模式状态机: 0=正常运行, 1=调秒, 2=调分, 3=调时
reg [1:0] set_mode;
//-----------------------------------------------------------------------------
//--  扩展功能寄存器
//-----------------------------------------------------------------------------
// 闪烁控制 (调整模式下闪烁显示)
reg [23:0] blink_cnt;          // 闪烁计数器
reg blink_on;                   // 闪烁状态 (1=显示, 0=熄灭)
parameter BLINK_PERIOD = 24'd12_500_000;  // 250ms 闪烁周期 @50MHz
// 整点报时控制
reg [3:0] chime_cnt;           // 整点报时闪烁次数计数
reg [24:0] chime_timer;        // 整点报时闪烁定时器
reg chime_active;              // 整点报时激活标志
reg chime_blink;               // 整点报时闪烁状态
parameter CHIME_BLINK_TIME = 25'd15_000_000;  // 300ms @50MHz
parameter CHIME_TIMES = 4'd6;  // 闪烁 6 次
// 闹钟功能
reg [5:0] alarm_h;             // 闹钟-小时 (0-23)
reg [5:0] alarm_m;             // 闹钟-分钟 (0-59)
reg alarm_set_mode;            // 闹钟设置模式标志 (1=设置闹钟)
reg [1:0] alarm_set_pos;       // 闹钟设置位置 (0=分, 1=时)
reg alarm_enabled;             // 闹钟使能
reg alarm_triggered;           // 闹钟触发标志
reg [23:0] alarm_flash_cnt;    // 闹钟闪烁计数器
reg alarm_flash;               // 闹钟闪烁状态
reg [28:0] alarm_timeout_cnt;  // 闹钟超时计数器
parameter ALARM_FLASH_PERIOD = 24'd6_250_000;  // 125ms 快速闪烁 @50MHz
parameter ALARM_TIMEOUT = 29'd500_000_000;     // 10 秒超时 @50MHz
```

```
// 长按检测 (用于进入闹钟设置)
```

```
reg [25:0] long_press_cnt;     // 长按计数器
reg long_press_pulse;          // 长按脉冲
parameter LONG_PRESS_TIME = 26'd50_000_000;  // 1 秒长按 @50MHz
//-----------------------------------------------------------------------------
//--  蜂鸣器控制 (闹钟触发和整点报时时响)
//-----------------------------------------------------------------------------
// 蜂鸣器频率产生 (约 2kHz 方波，适用于有源和无源蜂鸣器)
reg [15:0] beep_cnt;           // 蜂鸣器频率计数器
reg beep_tone;                 // 蜂鸣器音调信号
parameter BEEP_FREQ_CNT = 16'd12500;  // 50MHz/12500/2 = 2kHz
// 两级同步
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        k1_sync0 <= 1'b1; k1_sync1 <= 1'b1;
        k2_sync0 <= 1'b1; k2_sync1 <= 1'b1;
    end else begin
        k1_sync0 <= FLAG1; k1_sync1 <= k1_sync0;
        k2_sync0 <= FLAG2; k2_sync1 <= k2_sync0;
    end
end
// 消抖 + 产生按下脉冲（高->低稳定翻转产生 1 个 CLK 周期脉冲）
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        k1_cnt <= 20'd0; k1_level <= 1'b1; k1_level_prev <= 1'b1; mode_pulse <= 1'b0;
    end else begin
        mode_pulse <= 1'b0;
        if(k1_sync1 == k1_level) begin
            k1_cnt <= 20'd0;
        end else begin
            if(k1_cnt >= DB_CNT_MAX-1) begin
                // 先判断脉冲（使用旧值），再更新状态
                if(k1_level==1'b1 && k1_sync1==1'b0) mode_pulse <= 1'b1; // 按下
                k1_level_prev <= k1_level;
                k1_level <= k1_sync1;
                k1_cnt <= 20'd0;
            end else begin
                k1_cnt <= k1_cnt + 20'd1;
            end
        end
    end
end
```

```
// KEY2 消抖 + 产生按下脉冲
always @(posedge CLK_50M or negedge RST_N) begin
```

```
        if(!RST_N) begin
            k2_cnt <= 20'd0; k2_level <= 1'b1; k2_level_prev <= 1'b1; inc_pulse <= 1'b0;
        end else begin
            inc_pulse <= 1'b0;
            if(k2_sync1 == k2_level) begin
                k2_cnt <= 20'd0;
            end else begin
                if(k2_cnt >= DB_CNT_MAX-1) begin
                    // 先判断脉冲（使用旧值），再更新状态
                    // 按下瞬间产生脉冲（在调整模式或闹钟设置模式下有效，或取消闹钟）
                    if(k2_level==1'b1 && k2_sync1==1'b0 && (set_mode != 2'd0 || alarm_set_mode ||
alarm_triggered))
                        inc_pulse <= 1'b1;
                    k2_level_prev <= k2_level;
                    k2_level <= k2_sync1;
                    k2_cnt <= 20'd0;
                end else begin
                    k2_cnt <= k2_cnt + 20'd1;
                end
            end
        end
    end
    // 模式切换状态机: 0->正常, 1->调秒, 2->调分, 3->调时, 再按回 0
    // 闹钟设置模式下禁用时间调整
    always @(posedge CLK_50M or negedge RST_N) begin
        if(!RST_N)
            set_mode <= 2'd0;
        else if(alarm_set_mode)
            set_mode <= 2'd0;  // 闹钟设置模式下强制为正常模式
        else if(mode_pulse && !alarm_set_mode && !alarm_triggered)  // 非闹钟模式下切换
            set_mode <= set_mode + 2'd1;  // 0->1->2->3->0 循环
    end
    // 闹钟设置位置切换 (闹钟设置模式下短按 KEY1 切换时/分)
    always @(posedge CLK_50M or negedge RST_N) begin
        if(!RST_N)
            alarm_set_pos <= 2'd0;
        else if(!alarm_set_mode)
            alarm_set_pos <= 2'd0;  // 退出闹钟设置时复位
        else if(mode_pulse && alarm_set_mode)
            alarm_set_pos <= (alarm_set_pos == 2'd1) ? 2'd0 : 2'd1;  // 0->1->0 (分->时->分 循环)
    end
    //-----------------------------------------------------------------------
    //--  闪烁控制逻辑 (调整模式和闹钟设置模式下闪烁)
    //-----------------------------------------------------------------------
```

```
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        blink_cnt <= 24'd0;
        blink_on <= 1'b1;
    end else if(set_mode != 2'd0 || alarm_set_mode) begin
        // 调整模式或闹钟设置模式下启用闪烁
        if(blink_cnt >= BLINK_PERIOD - 1) begin
            blink_cnt <= 24'd0;
            blink_on <= ~blink_on;
        end else begin
            blink_cnt <= blink_cnt + 24'd1;
        end
    end else begin
        blink_cnt <= 24'd0;
        blink_on <= 1'b1;  // 正常模式下始终显示
    end
end
//-----------------------------------------------------------------------------
//--  整点报时逻辑
//-----------------------------------------------------------------------------
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        chime_active <= 1'b0;
        chime_cnt <= 4'd0;
        chime_timer <= 25'd0;
        chime_blink <= 1'b1;
    end else begin
        // 检测整点 (分秒都为0时触发, 闹钟设置模式下不触发)
        if(set_mode == 2'd0 && cnt_s == 6'd0 && cnt_m == 6'd0 &&
           cnt_1s == SET_TIME_1S_FWD - 1 && !chime_active && !alarm_set_mode) begin
            chime_active <= 1'b1;
            chime_cnt <= 4'd0;
            chime_timer <= 25'd0;
            chime_blink <= 1'b0;
        end
        // 整点报时闪烁过程
        else if(chime_active) begin
            if(chime_timer >= CHIME_BLINK_TIME - 1) begin
                chime_timer <= 25'd0;
                chime_blink <= ~chime_blink;
                if(chime_blink == 1'b1) begin  // 完成一次亮灭周期
                    chime_cnt <= chime_cnt + 4'd1;
                end
                if(chime_cnt >= CHIME_TIMES) begin
```

```
                    chime_active <= 1'b0;
                    chime_blink <= 1'b1;
                end
            end else begin
                chime_timer <= chime_timer + 25'd1;
            end
        end
    end
end
//-----------------------------------------------------------------------
//--  闹钟功能逻辑
//-----------------------------------------------------------------------
// 长按 KEY1 检测 (用于进入/退出闹钟设置模式)
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        long_press_cnt <= 26'd0;
        long_press_pulse <= 1'b0;
    end else begin
        long_press_pulse <= 1'b0;
        if(k1_level == 1'b0 && set_mode == 2'd0) begin  // 按键按下中且在正常模式
            if(long_press_cnt >= LONG_PRESS_TIME - 1) begin
                if(long_press_cnt == LONG_PRESS_TIME - 1)
                    long_press_pulse <= 1'b1;
                long_press_cnt <= LONG_PRESS_TIME;  // 保持，防止重复触发
            end else begin
                long_press_cnt <= long_press_cnt + 26'd1;
            end
        end else begin
            long_press_cnt <= 26'd0;
        end
    end
end
// 闹钟设置模式切换
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_set_mode <= 1'b0;
    end else if(long_press_pulse && !alarm_triggered) begin
        alarm_set_mode <= ~alarm_set_mode;
    end else if(long_press_pulse && alarm_triggered) begin
        alarm_set_mode <= 1'b0;  // 闹钟响时长按不进入设置
    end
end
```

```
// 闹钟使能控制 (退出闹钟设置模式时自动使能)
```

```
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_enabled <= 1'b0;
    end else if(alarm_set_mode && long_press_pulse) begin
        alarm_enabled <= 1'b1;  // 退出设置时使能闹钟
    end
end
// 闹钟时间设置 - 分钟
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_m <= 6'd0;
    end else if(alarm_set_mode && alarm_set_pos == 2'd0 && inc_pulse) begin
        if(alarm_m >= SIXTY - 1)
            alarm_m <= 6'd0;
        else
            alarm_m <= alarm_m + 6'd1;
    end
end
// 闹钟时间设置 - 小时
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_h <= 6'd0;
    end else if(alarm_set_mode && alarm_set_pos == 2'd1 && inc_pulse) begin
        if(alarm_h >= TWENTYFORE - 1)
            alarm_h <= 6'd0;
        else
            alarm_h <= alarm_h + 6'd1;
    end
end
// 闹钟触发检测
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_triggered <= 1'b0;
    end else if(alarm_triggered && (mode_pulse || inc_pulse)) begin
        alarm_triggered <= 1'b0;  // 任意按键取消闹钟
    end else if(alarm_triggered && alarm_timeout_cnt >= ALARM_TIMEOUT - 1) begin
        alarm_triggered <= 1'b0;  // 超时自动停止
    end else if(alarm_enabled && !alarm_set_mode && set_mode == 2'd0 &&
                cnt_h == alarm_h && cnt_m == alarm_m && cnt_s == 6'd0 &&
                cnt_1s == SET_TIME_1S_FWD - 1) begin
        alarm_triggered <= 1'b1;  // 触发闹钟
    end
end
```

```
// 闹钟超时计数器
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_timeout_cnt <= 29'd0;
    end else if(!alarm_triggered) begin
        alarm_timeout_cnt <= 29'd0;  // 未触发时清零
    end else if(alarm_timeout_cnt < ALARM_TIMEOUT) begin
        alarm_timeout_cnt <= alarm_timeout_cnt + 29'd1;
    end
end
// 闹钟闪烁控制
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        alarm_flash_cnt <= 24'd0;
        alarm_flash <= 1'b1;
    end else if(alarm_triggered) begin
        if(alarm_flash_cnt >= ALARM_FLASH_PERIOD - 1) begin
            alarm_flash_cnt <= 24'd0;
            alarm_flash <= ~alarm_flash;
        end else begin
            alarm_flash_cnt <= alarm_flash_cnt + 24'd1;
        end
    end else begin
        alarm_flash_cnt <= 24'd0;
        alarm_flash <= 1'b1;
    end
end
//-----------------------------------------------------------------------
//--  蜂鸣器控制逻辑 (闹钟触发或整点报时时响)
//-----------------------------------------------------------------------
// 产生蜂鸣器音调 (约 2kHz 方波)
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        beep_cnt <= 16'd0;
        beep_tone <= 1'b0;
    end else begin
        if(beep_cnt >= BEEP_FREQ_CNT - 1) begin
            beep_cnt <= 16'd0;
            beep_tone <= ~beep_tone;
        end else begin
            beep_cnt <= beep_cnt + 16'd1;
        end
    end
end
```

```
// 蜂鸣器输出控制
// 闹钟触发时：持续响（可用按键取消）
// 整点报时时：间歇响（跟随闪烁节奏）
always @(posedge CLK_50M or negedge RST_N) begin
    if(!RST_N) begin
        BEEP <= 1'b0;
    end else if(alarm_triggered) begin
        // 闹钟触发：蜂鸣器持续发出声音
        BEEP <= beep_tone;
    end else if(chime_active) begin
        // 整点报时：蜂鸣器跟随闪烁节奏响（chime_blink 为 0 时响）
        BEEP <= ~chime_blink & beep_tone;
    end else begin
        // 正常模式：蜂鸣器关闭
        BEEP <= 1'b0;
    end
end
//-----------------------------------------------------------------------------
//--    内部端口声明
//-----------------------------------------------------------------------------
reg         [15:0]  time_cnt;           //用来控制数码管闪烁频率的定时计数器
reg         [15:0]  time_cnt_n;         //time_cnt 的下一个状态
reg         [ 2:0]  led_cnt;                //用来控制数码管亮灭及显示数据的显示计数器
reg         [ 2:0]  led_cnt_n;          //led_cnt 的下一个状态
// cnt_1s, cnt_s, cnt_m, cnt_h 已在前面声明
//reg         [3:0] cnt_12s;
//reg         [7:0] cnt_12s_8bit;
reg         [7:0]   seg0,seg1;         //秒
reg         [7:0]   seg2,seg3;         //分
reg         [7:0]   seg4,seg5;         //时
wire         [3:0]  ones_bit;
wire         [3:0]  tens_bit;
//二进制转 bcd 十进制
reg [5:0] s_ones;
reg [5:0] s_tens;
reg [5:0] m_ones;
reg [5:0] m_tens;
reg [5:0] h_ones;
reg [5:0] h_tens;
wire [5:0] s_ones_bit;
wire [5:0] s_tens_bit;
wire [5:0] m_ones_bit;
wire [5:0] m_tens_bit;
wire [5:0] h_ones_bit;
```

```
wire [5:0] h_tens_bit;
// 闹钟 BCD 转换
reg [5:0] alarm_m_ones, alarm_m_tens;
reg [5:0] alarm_h_ones, alarm_h_tens;
wire [5:0] alarm_m_ones_bit, alarm_m_tens_bit;
wire [5:0] alarm_h_ones_bit, alarm_h_tens_bit;
// 最终显示值 (用于在时钟/闹钟设置之间切换)
wire [5:0] disp_s_ones, disp_s_tens;
wire [5:0] disp_m_ones, disp_m_tens;
wire [5:0] disp_h_ones, disp_h_tens;
integer i;

//-----------------------------------------------------------------------------
//--    逻辑功能实现
//-----------------------------------------------------------------------------
//时钟
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)
        cnt_1s <= 28'd0;
    else if(set_mode != 2'd0)  // 调整模式下暂停走时
        cnt_1s <= 28'd0;
    else if(cnt_1s == SET_TIME_1S - 1)
        cnt_1s <= 28'd0;
    else
        cnt_1s <= cnt_1s + 28'b1;
end
//秒 实现
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)
        cnt_s <= state_max_yellow;
    else if(set_mode == 2'd1 && inc_pulse) begin  // 调秒模式
        if(cnt_s == SIXTY - 1)
            cnt_s <= 6'd0;
        else
            cnt_s <= cnt_s + 6'd1;
    end
    else if(set_mode == 2'd0) begin  // 正常运行模式
        if(cnt_s == SIXTY-1 && cnt_1s == SET_TIME_1S - 1)
            cnt_s <= 6'd0;
        else if(cnt_1s == SET_TIME_1S - 1)
            cnt_s <= cnt_s + 6'd1;
    end
```

```
end
//分钟 实现
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)
        cnt_m <= state_max_yellow;
    else if(set_mode == 2'd2 && inc_pulse) begin  // 调分模式
        if(cnt_m == SIXTY - 1)
            cnt_m <= 6'd0;
        else
            cnt_m <= cnt_m + 6'd1;
    end
    else if(set_mode == 2'd0) begin  // 正常运行模式
        if(cnt_m == SIXTY-1 && cnt_s == SIXTY-1 && cnt_1s == SET_TIME_1S - 1)
            cnt_m <= 6'd0;
        else if(cnt_s == SIXTY-1 && cnt_1s == SET_TIME_1S - 1)
            cnt_m <= cnt_m + 6'd1;
    end
end
//小时 实现
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)
        cnt_h <= state_max_yellow;
    else if(set_mode == 2'd3 && inc_pulse) begin  // 调时模式
        if(cnt_h == TWENTYFORE - 1)
            cnt_h <= 6'd0;
        else
            cnt_h <= cnt_h + 6'd1;
    end
    else if(set_mode == 2'd0) begin  // 正常运行模式
        if(cnt_h == TWENTYFORE-1 && cnt_m == SIXTY-1 && cnt_s == SIXTY-1 && cnt_1s == SET_TIME_1S - 1)
            cnt_h <= 6'd0;
        else if(cnt_m == SIXTY-1 && cnt_s == SIXTY-1 && cnt_1s == SET_TIME_1S - 1)
            cnt_h <= cnt_h + 6'd1;
    end
end
/////////////进制转换/////////////////////////////////////////////////////
//秒
always @(*) begin
 begin
    s_ones      = 4'd0;
    s_tens      = 4'd0;
```

```
 end

    for(i = 5; i >= 0; i = i - 1) begin
        if (s_ones >= 4'd5)         s_ones = s_ones + 4'd3;
        if (s_tens >= 4'd5)         s_tens = s_tens + 4'd3;
        s_tens   = {2'b00, s_tens[2:0], s_ones[3]};
        s_ones   = {2'b00, s_ones[2:0], cnt_s[i]};
    end
end
assign s_tens_bit = s_tens;
assign s_ones_bit = s_ones;
//分
always @(*) begin
 begin
    m_ones      = 4'd0;
    m_tens      = 4'd0;
 end

    for(i = 5; i >= 0; i = i - 1) begin
        if (m_ones >= 4'd5)         m_ones = m_ones + 4'd3;
        if (m_tens >= 4'd5)         m_tens = m_tens + 4'd3;
        m_tens   = {2'b00, m_tens[2:0], m_ones[3]};
        m_ones   = {2'b00, m_ones[2:0], cnt_m[i]};
    end
end
assign m_tens_bit = m_tens;
assign m_ones_bit = m_ones;
//时
always @(*) begin
 begin
    h_ones      = 4'd0;
    h_tens      = 4'd0;
 end

    for(i = 5; i >= 0; i = i - 1) begin
        if (h_ones >= 4'd5)         h_ones = h_ones + 4'd3;
        if (h_tens >= 4'd5)         h_tens = h_tens + 4'd3;
        h_tens   = {2'b00, h_tens[2:0], h_ones[3]};
        h_ones   = {2'b00, h_ones[2:0], cnt_h[i]};
    end
end
assign h_tens_bit = h_tens;
assign h_ones_bit = h_ones;
```

```
/////////////闹钟进制转换//////////////////////////////////////////////
// 闹钟-分钟 BCD 转换
always @(*) begin
    alarm_m_ones = 4'd0;
    alarm_m_tens = 4'd0;
    for(i = 5; i >= 0; i = i - 1) begin
        if (alarm_m_ones >= 4'd5) alarm_m_ones = alarm_m_ones + 4'd3;
        if (alarm_m_tens >= 4'd5) alarm_m_tens = alarm_m_tens + 4'd3;
        alarm_m_tens = {2'b00, alarm_m_tens[2:0], alarm_m_ones[3]};
        alarm_m_ones = {2'b00, alarm_m_ones[2:0], alarm_m[i]};
    end
end
assign alarm_m_tens_bit = alarm_m_tens;
assign alarm_m_ones_bit = alarm_m_ones;
// 闹钟-小时 BCD 转换
always @(*) begin
    alarm_h_ones = 4'd0;
    alarm_h_tens = 4'd0;
    for(i = 5; i >= 0; i = i - 1) begin
        if (alarm_h_ones >= 4'd5) alarm_h_ones = alarm_h_ones + 4'd3;
        if (alarm_h_tens >= 4'd5) alarm_h_tens = alarm_h_tens + 4'd3;
        alarm_h_tens = {2'b00, alarm_h_tens[2:0], alarm_h_ones[3]};
        alarm_h_ones = {2'b00, alarm_h_ones[2:0], alarm_h[i]};
    end
end
assign alarm_h_tens_bit = alarm_h_tens;
assign alarm_h_ones_bit = alarm_h_ones;
/////////////显示值选择 (时钟/闹钟设置切换)//////////////////////////////
// 闹钟设置模式显示闹钟时间，否则显示当前时间
assign disp_s_ones = alarm_set_mode ? 4'd0 : s_ones_bit;  // 闹钟设置时秒位显示 00
assign disp_s_tens = alarm_set_mode ? 4'd0 : s_tens_bit;
assign disp_m_ones = alarm_set_mode ? alarm_m_ones_bit : m_ones_bit;
assign disp_m_tens = alarm_set_mode ? alarm_m_tens_bit : m_tens_bit;
assign disp_h_ones = alarm_set_mode ? alarm_h_ones_bit : h_ones_bit;
assign disp_h_tens = alarm_set_mode ? alarm_h_tens_bit : h_tens_bit;
//////////////////////////////////////////////////////////////////////////
//////////////////数码管段码值的给出////////////////////////////////////
//秒位 (闹钟设置模式显示 00) 实现
always @(*) begin
if(!RST_N) begin
    seg0 <= SEG_CODE_0;
end else
    case(disp_s_ones)
    'd0: seg0 <= SEG_CODE_0;
```

```
        'd1: seg0 <= SEG_CODE_1;
        'd2: seg0 <= SEG_CODE_2;
        'd3: seg0 <= SEG_CODE_3;
        'd4: seg0 <= SEG_CODE_4;
        'd5: seg0 <= SEG_CODE_5;
        'd6: seg0 <= SEG_CODE_6;
        'd7: seg0 <= SEG_CODE_7;
        'd8: seg0 <= SEG_CODE_8;
        'd9: seg0 <= SEG_CODE_9;
        default:seg0 <= SEG_CODE_0;
        endcase
    end
    always @(*) begin
    if(!RST_N) begin
        seg1 <= SEG_CODE_0;
    end else
        case(disp_s_tens)
        'd0: seg1 <= SEG_CODE_0;
        'd1: seg1 <= SEG_CODE_1;
        'd2: seg1 <= SEG_CODE_2;
        'd3: seg1 <= SEG_CODE_3;
        'd4: seg1 <= SEG_CODE_4;
        'd5: seg1 <= SEG_CODE_5;
        'd6: seg1 <= SEG_CODE_6;
        'd7: seg1 <= SEG_CODE_7;
        'd8: seg1 <= SEG_CODE_8;
        'd9: seg1 <= SEG_CODE_9;
        default:seg1 <= SEG_CODE_0;
        endcase
    end
    //分位 (闹钟设置模式显示闹钟分钟) 实现
    always @(*) begin
    if(!RST_N) begin
        seg2 <= SEG_CODE_0;
    end else
        case(disp_m_ones)
        'd0: seg2 <= SEG_CODE_0;
        'd1: seg2 <= SEG_CODE_1;
        'd2: seg2 <= SEG_CODE_2;
        'd3: seg2 <= SEG_CODE_3;
        'd4: seg2 <= SEG_CODE_4;
        'd5: seg2 <= SEG_CODE_5;
        'd6: seg2 <= SEG_CODE_6;
        'd7: seg2 <= SEG_CODE_7;
```

```
        'd8: seg2 <= SEG_CODE_8;
        'd9: seg2 <= SEG_CODE_9;
        default:seg2 <= SEG_CODE_0;
        endcase
    end
    always @(*) begin
    if(!RST_N) begin
        seg3 <= SEG_CODE_0;
    end else
        case(disp_m_tens)
        'd0: seg3 <= SEG_CODE_0;
        'd1: seg3 <= SEG_CODE_1;
        'd2: seg3 <= SEG_CODE_2;
        'd3: seg3 <= SEG_CODE_3;
        'd4: seg3 <= SEG_CODE_4;
        'd5: seg3 <= SEG_CODE_5;
        'd6: seg3 <= SEG_CODE_6;
        'd7: seg3 <= SEG_CODE_7;
        'd8: seg3 <= SEG_CODE_8;
        'd9: seg3 <= SEG_CODE_9;
        default:seg3 <= SEG_CODE_0;
        endcase
    end
    // 小时位 (闹钟设置模式显示闹钟小时) 实现
    always @(*) begin
    if(!RST_N) begin
        seg4 <= SEG_CODE_0;
    end else
        case(disp_h_ones)
        'd0: seg4 <= SEG_CODE_0;
        'd1: seg4 <= SEG_CODE_1;
        'd2: seg4 <= SEG_CODE_2;
        'd3: seg4 <= SEG_CODE_3;
        'd4: seg4 <= SEG_CODE_4;
        'd5: seg4 <= SEG_CODE_5;
        'd6: seg4 <= SEG_CODE_6;
        'd7: seg4 <= SEG_CODE_7;
        'd8: seg4 <= SEG_CODE_8;
        'd9: seg4 <= SEG_CODE_9;
        default:seg4 <= SEG_CODE_0;
        endcase
    end
```

```
    always @(*) begin
```

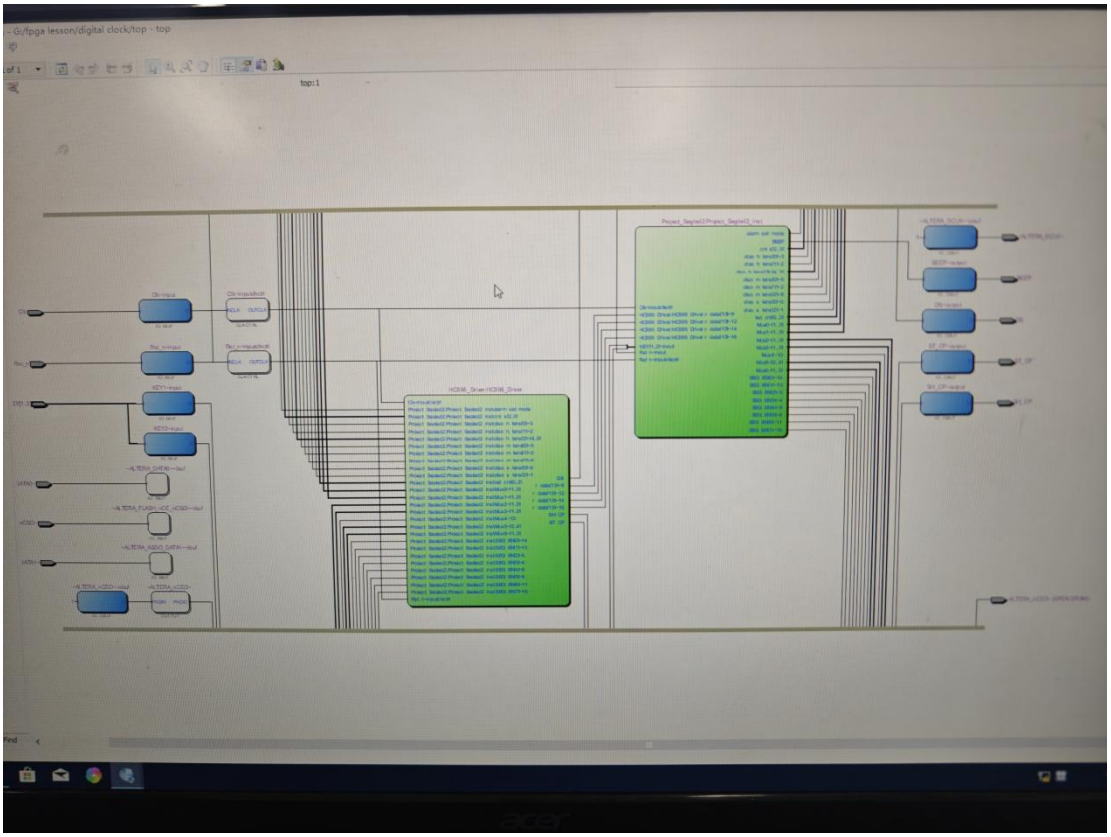
```
if(!RST_N) begin
    seg5 <= SEG_CODE_0;
end else
    case(disp_h_tens)
    'd0: seg5 <= SEG_CODE_0;
    'd1: seg5 <= SEG_CODE_1;
    'd2: seg5 <= SEG_CODE_2;
    'd3: seg5 <= SEG_CODE_3;
    'd4: seg5 <= SEG_CODE_4;
    'd5: seg5 <= SEG_CODE_5;
    'd6: seg5 <= SEG_CODE_6;
    'd7: seg5 <= SEG_CODE_7;
    'd8: seg5 <= SEG_CODE_8;
    'd9: seg5 <= SEG_CODE_9;
    default:seg5 <= SEG_CODE_0;
    endcase
end
//////////////////////////////////////////////////////////////////////////////////
///////这里是数码管每隔 1ms 的计数//////////////////////////////////////////////////////
//时序电路,用来给 time_cnt 寄存器赋值
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)                                  //判断复位
        time_cnt <= 16'h0;                  //初始化 time_cnt 值
    else
        time_cnt <= time_cnt_n;             //用来给 time_cnt 赋值
end
//组合电路,实现 1ms 的定时计数器
always @ (*)
begin
    if(time_cnt == SET_TIME_1MS)            //判断 1ms 时间
        time_cnt_n = 16'h0;                 //如果到达 1ms,定时计数器将会被清零
    else
        time_cnt_n = time_cnt + 16'd1;  //如果未到 1ms,定时计数器将会继续累加
end
//////////////////////////////////////////////////////////////////////////////////
////////////这里是数码管依次选通的计数//////////////////////////////////////////////
//时序电路,用来给 led_cnt 寄存器赋值
always @ (posedge CLK_50M or negedge RST_N)
begin
    if(!RST_N)                                  //判断复位
        led_cnt <= 3'h0;                        //初始化 led_cnt 值
    else
        led_cnt <= led_cnt_n;               //用来给 led_cnt 赋值
```

```
    end
    //组合电路,判断时间，实现控制显示计数器累加
    always @ (*)
    begin
        if(time_cnt == SET_TIME_1MS)            //判断 1ms 时间
            led_cnt_n = led_cnt + 1'h1;     //如果到达 1ms,计数器进行累加
        else
            led_cnt_n = led_cnt;                    //如果未到 1ms,计数器保持不变
    end
    ////////////////////////////////////////////////////////////////////////////
    //组合电路,实现数码管的数字显示
    // 显示格式: HH-MM-SS (位置从右到左: 0=秒个位, 1=秒十位, 2=横线, 3=分个位, 4=分十位, 5=横线, 6=时个位, 7=时十
位)
    always @ (*)
    begin
        case (led_cnt)
            3'b000 : SEG_DATA = seg0;         // 秒个位
            3'b001 : SEG_DATA = seg1;         // 秒十位
            3'b010 : SEG_DATA = SEG_CODE_DASH; // 横线分隔符 (分秒之间)
            3'b011 : SEG_DATA = seg2;         // 分个位
            3'b100 : SEG_DATA = seg3;         // 分十位
            3'b101 : SEG_DATA = SEG_CODE_DASH; // 横线分隔符 (时分之间)
            3'b110 : SEG_DATA = seg4;         // 时个位
            3'b111 : SEG_DATA = seg5;         // 时十位
            default: SEG_DATA = SEG_CODE_0;
        endcase
    end
    //组合电路,控制数码管亮灭 (含闪烁效果、整点报时和闹钟提示)
    // 辅助信号: 判断当前位是否需要闪烁
    // 时间调整模式下的闪烁（非闹钟设置模式）
    wire sec_blink  = (set_mode == 2'd1) && !alarm_set_mode && !blink_on;  // 调秒模式闪烁
    wire min_blink  = (set_mode == 2'd2) && !alarm_set_mode && !blink_on;  // 调分模式闪烁
    wire hour_blink = (set_mode == 2'd3) && !alarm_set_mode && !blink_on;  // 调时模式闪烁
    // 闹钟设置模式下的闪烁
    wire alarm_min_blink  = alarm_set_mode && (alarm_set_pos == 2'd0) && !blink_on;  // 闹钟调分模式闪烁
    wire alarm_hour_blink = alarm_set_mode && (alarm_set_pos == 2'd1) && !blink_on;  // 闹钟调时模式闪烁
    wire chime_off  = chime_active && !chime_blink;     // 整点报时熄灭
    wire alarm_off  = alarm_triggered && !alarm_flash;  // 闹钟触发快速闪烁
    // 显示格式: HH-MM-SS (位置从右到左: 0=秒个位, 1=秒十位, 2=横线, 3=分个位, 4=分十位, 5=横线, 6=时个位, 7=时十
位)
    always @ (*)
    begin
        // 闹钟触发时全部快速闪烁 (最高优先级)
        if(alarm_off) begin
```

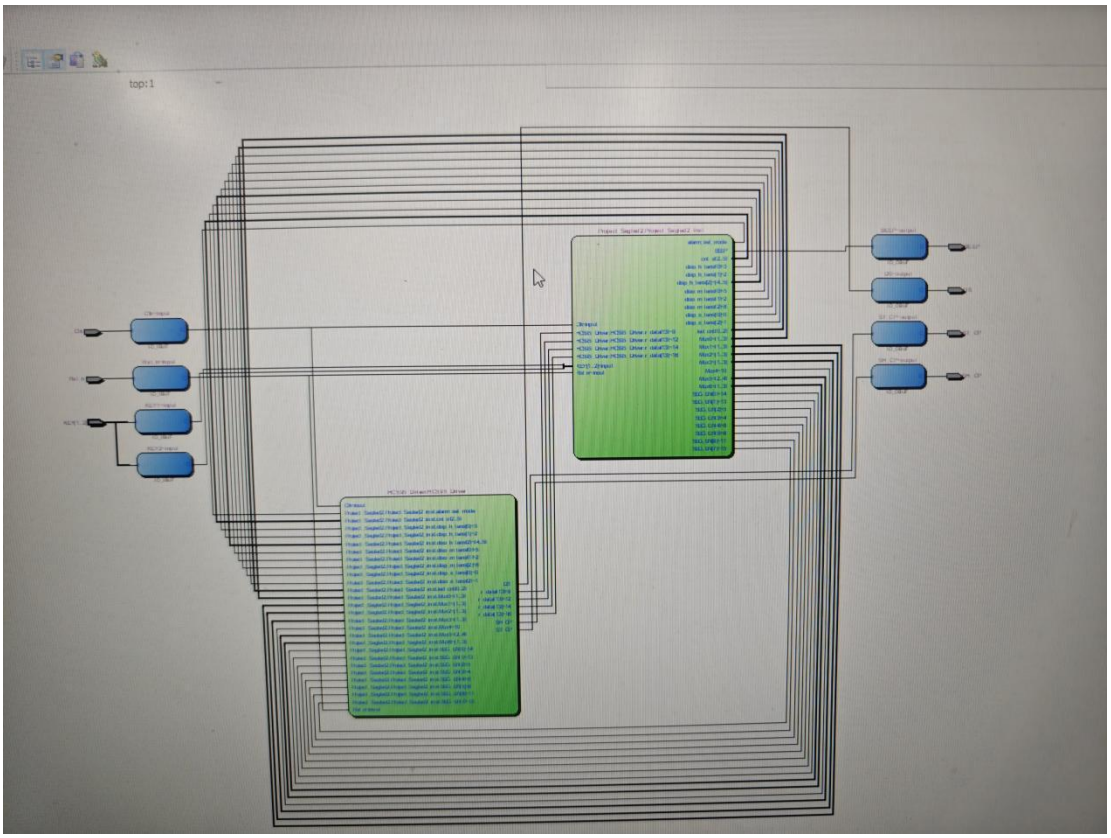
```
            SEG_EN = 8'b0000_0000;  // 闹钟触发时熄灭
        end
        // 整点报时次优先级
        else if(chime_off && !alarm_set_mode) begin
            SEG_EN = 8'b0000_0000;  // 整点报时时全部熄灭
        end else begin
            case (led_cnt)
                3'b000 : SEG_EN = sec_blink ? 8'b0000_0000 : 8'b0000_0001;  // 秒个位
                3'b001 : SEG_EN = sec_blink ? 8'b0000_0000 : 8'b0000_0010;  // 秒十位
                3'b010 : SEG_EN = 8'b0000_0100;  // 横线分隔符 (分秒之间) - 始终显示
                3'b011 : SEG_EN = (min_blink || alarm_min_blink) ? 8'b0000_0000 : 8'b0000_1000;  // 分个位
                3'b100 : SEG_EN = (min_blink || alarm_min_blink) ? 8'b0000_0000 : 8'b0001_0000;  // 分十位
                3'b101 : SEG_EN = 8'b0010_0000;  // 横线分隔符 (时分之间) - 始终显示
                3'b110 : SEG_EN = (hour_blink || alarm_hour_blink) ? 8'b0000_0000 : 8'b0100_0000;  // 时个位
                3'b111 : SEG_EN = (hour_blink || alarm_hour_blink) ? 8'b0000_0000 : 8'b1000_0000;  // 时十位
                default: SEG_EN = 8'b0000_0000;
            endcase
        end
    end
    endmodule
```

2.2、RTL 电路（10 分）

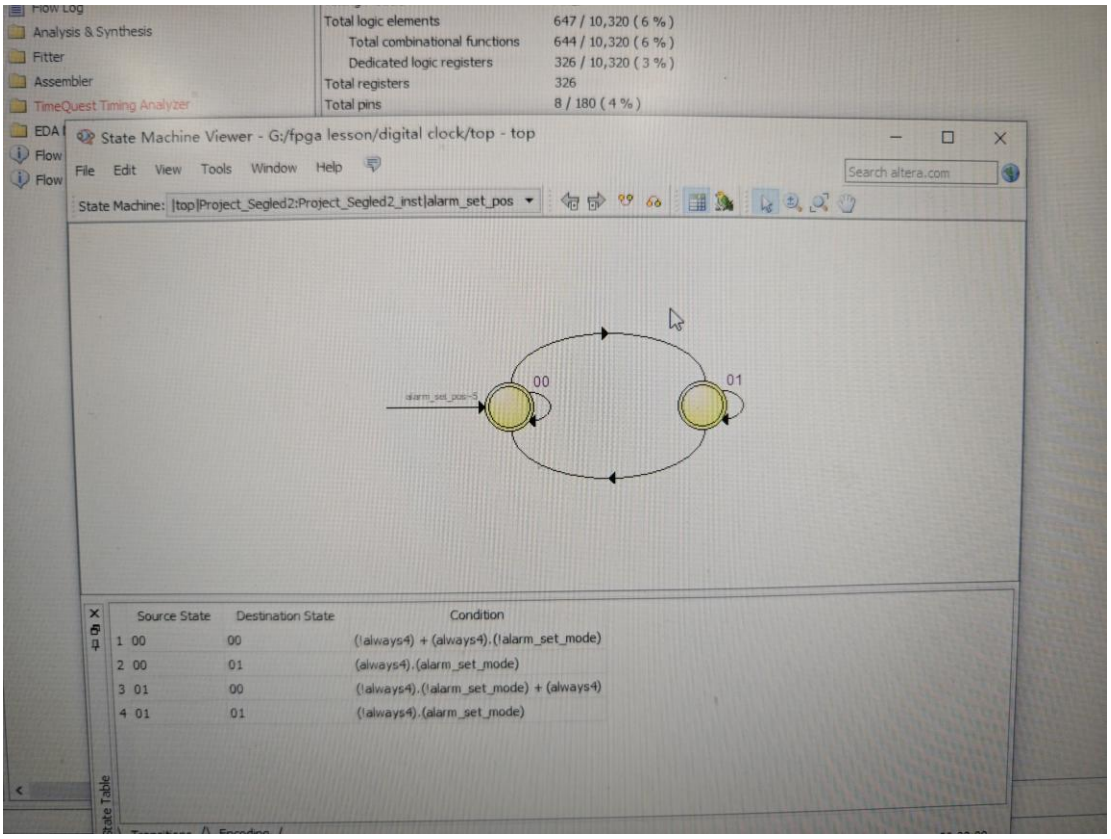
图一到四为在 Quartus 软件中对设计进行综合后生成的 RTL 原理图、模块及其信号连接关系图、状态机状态转移图及 RTL 顶层结构图。



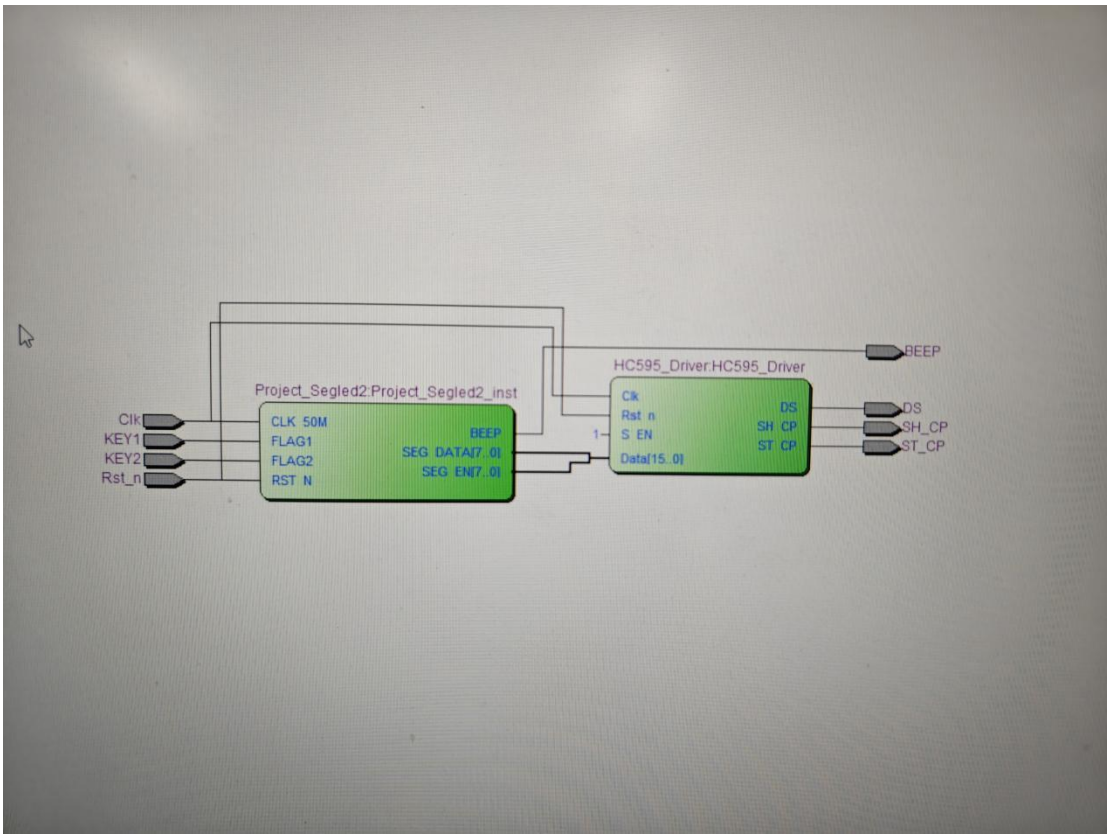
图一 电路拓扑图



图二 RTL 原理图



图三 状态机状态转移图


Clk
KEY1
KEY2
Rst_n
Project_Segled2:Project_Segled2_inst
CLK_50M
FLAG1
FLAG2
RST_N
BEEP
SEG_DATA[7..0]
SEG_EN[7..0]
HC595_Driver:HC595_Driver
Clk
Rst_n
S_EN
Data[15..0]
DS
SH_CP
ST_CP
BEEP
DS
SH_CP
ST_CP

图四 RTL 顶层结构图

3、waveform 波形（10 分）

在 Quartus 软件中调出 Waveform 波形，对电子时钟系统进行初步功能仿真，其仿真结果如图所示。

4.1、testbench 代码（10 分）

```
仿真验证

为验证设计功能的正确性，编写了 Verilog 测试平台（Testbench）对数字时钟模块进行仿真测试。
**仿真文件：** tb_time_led.v
**仿真环境配置：**
- 时间精度：1ns/1ns
- 系统时钟：50MHz（周期 20ns，通过 `#10 Clk = ~Clk` 实现）
**测试信号定义：**
| 信号名 | 类型 | 说明 |
|--------|------|------|
| Clk | reg | 50MHz 时钟激励 |
| Rst_n | reg | 复位信号 |
| STATE_KEY | reg | 模式切换按键（对应 KEY1） |
| SET_KEY | reg | 数值设置按键（对应 KEY2） |
| BEEP | wire | 蜂鸣器输出观测 |
| SH_CP/ST_CP/DS | wire | 74HC595 控制信号观测 |
**测试流程：**
```
时间轴 ──▶
0ns   200ns   1200ns   2200ns
| | | |
▼   ▼   ▼   ▼
初始化 → 复位拉低 → 复位释放 → 开始按键测试
         (1000ns)   (1000ns)
按键测试序列:
┌──┐
│ 1. 切换到时钟调节状态: STATE_KEY 按下→释放   │
│ 2. 连续两次 SET_KEY: 时间值+2   │
│ 3. 切换到分钟调节状态: STATE_KEY 按下→释放   │
│ 4. 连续两次 SET_KEY: 分钟值+2   │
│ 5. 切换到秒钟调节状态: STATE_KEY 按下→释放   │
│ 6. 连续两次 SET_KEY: 秒钟值+2   │
│ 7. 再次 STATE_KEY: 恢复正常计时状态   │
│ 8. 继续观察若干周期后 $stop 结束仿真   │
└──┘
```
**按键时序说明：**
- 每次按键保持低电平 2000ns（模拟按下）
- 按键释放后保持高电平 2000ns（模拟松开）
```

- 按键间隔足够消抖电路稳定

仿真验证要点：

1. 复位功能：复位后各计数器清零，显示 00-00-00
2. 模式切换：STATE_KEY 依次切换 正常→调时→调分→调秒→正常
3. 数值调整：在调整模式下 SET_KEY 使对应位递增
4. 输出观测：通过 SH_CP/ST_CP/DS 波形验证 74HC595 驱动时序

仿真结果预期：

- 各模式切换正常，状态机工作正确
- 按键消抖后产生有效脉冲
- 74HC595 串行输出波形符合时序要求

```
`timescale 1ns/1ns
module tb_time_led ();

reg Clk;
reg Rst_n;
reg STATE_KEY;
reg SET_KEY;
wire BEEP;
wire SH_CP;
wire ST_CP;
wire DS;
time_led time_led_inst(
    .Clk(Clk),
    .Rst_n(Rst_n),
    .STATE_KEY(STATE_KEY),
    .SET_KEY(SET_KEY),
    .BEEP(BEEP),
    .SH_CP(SH_CP),
    .ST_CP(ST_CP),
    .DS(DS)
);
initial begin
    Clk = 1'b0;
    forever #10 Clk = ~Clk;
end
initial begin
    Rst_n = 1'b1;
    STATE_KEY = 1'b1;
    SET_KEY = 1'b1;
    #200;

    Rst_n = 1'b0;
    #1000;
    Rst_n = 1'b1;
    #1000;
//切换到时钟调节状态
    STATE_KEY = 1'b0;
    #2000;
    STATE_KEY = 1'b1;
    #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
```

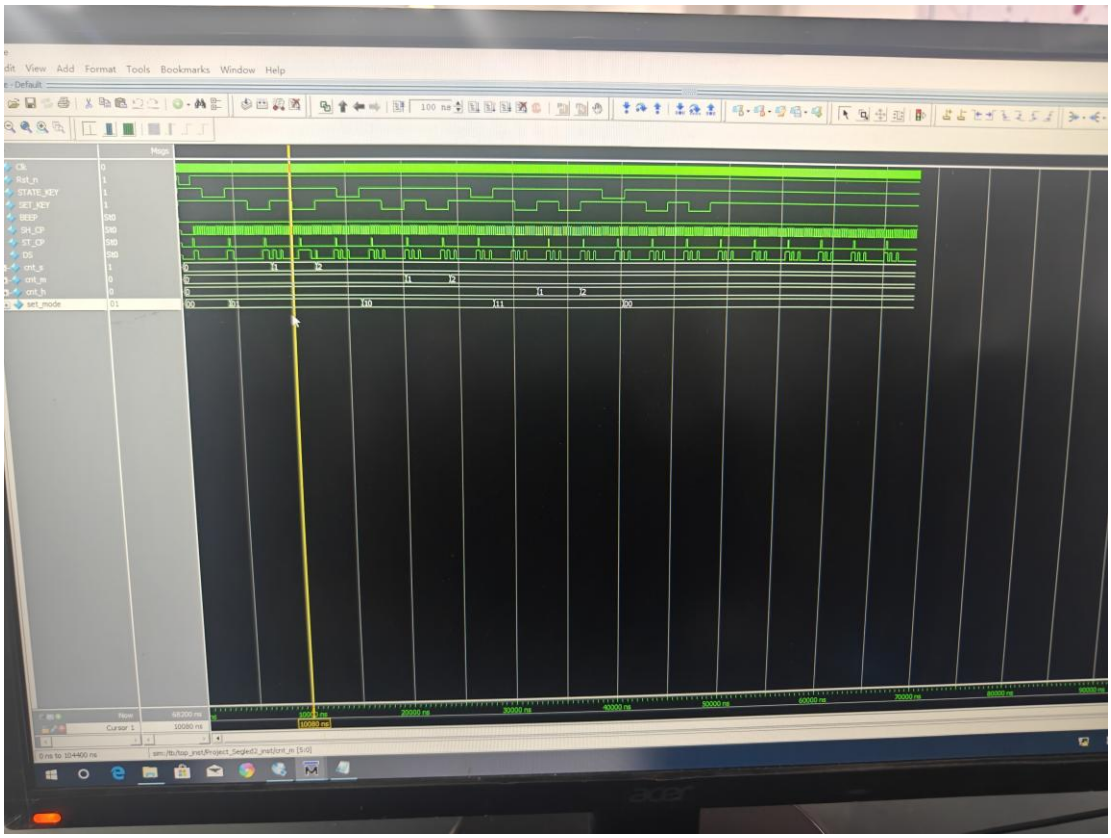
```
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
///切换到分钟调节状态
    STATE_KEY = 1'b0;
    #2000;
    STATE_KEY = 1'b1;
    #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
//切换到秒钟调节状态
    STATE_KEY = 1'b0;
    #2000;
    STATE_KEY = 1'b1;
    #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
    STATE_KEY = 1'b0;
    #2000;
    STATE_KEY = 1'b1;
    #2000;
//恢复到计时状态
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
   #2000;
    SET_KEY = 1'b0;
   #2000;
    SET_KEY = 1'b1;
```

```
    #20000;


     $stop;
end
endmodule
```

4.2、testbench 仿真波形（10 分）

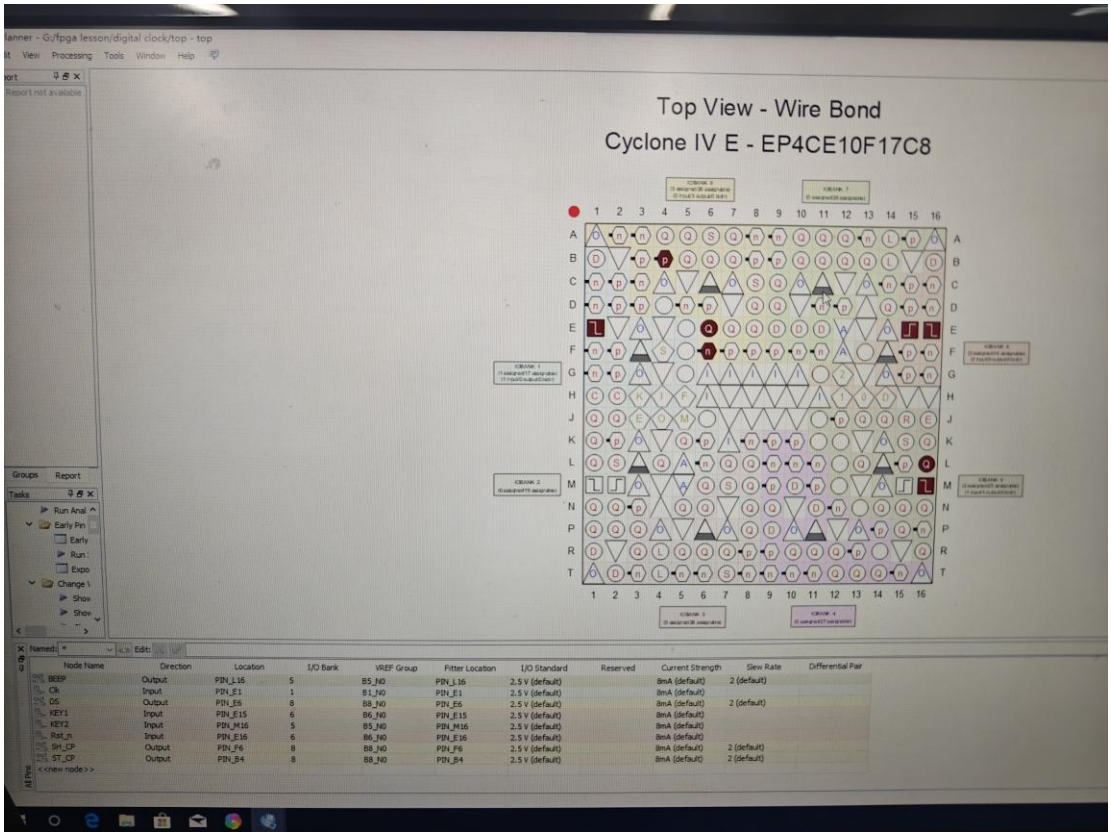
通过直接读取数字时钟模块中时、分、秒寄存器的数值进行仿真分析，在仿真过程中直接观测数字时钟模块内部时、分、秒寄存器的数值变化，结果完全正确，验证了时间计数逻辑的正确性。



5、FPGA 上板

（1）管脚分配（10 分）

将三个 HC595 芯片的相关控制与数据信号映射至对应的物理引脚，系统时钟采用板载 50 MHz 晶振，复位信号由按键三提供；按键一用于加操作控制，按键二用于状态切换控制，BEEP 信号连接至板载无源蜂鸣器。



（2）实现功能（10 分）（课堂检查）

本报告所实现的所有功能均已在上课由老师检查，功能已完全实现。

自由发挥（10 分）

在本实验的基础功能之上，进一步扩展并实现了以下功能：

闹钟功能

实现了可设定的闹钟功能，当系统时间到达指定的小时和分钟时，触发数码管闪屏提示，同时蜂鸣器持续鸣响约 10 秒，以提醒用户。

整点报时功能

实现了整点报时功能，在每个整点时刻自动触发数码管闪屏效果，蜂鸣器以间断方式鸣响约 5 秒，用于提示整点到达。

调整位闪烁功能

在时间调整模式下，被调整的时间位（小时或分钟）以闪烁方式显示，其余位保持正常显示状态，从而直观提示当前的调整对象，提高了系统的人机交互体验。